

How to Train a Jev-Like Model

Treating Typed Evaluation as One-Token Classification

Jioh Jung
ziozzang@skku.edu

September 21, 2026

Abstract

A Jev-like model does not need to learn to write an answer. It needs to place probability mass on one member of a declared finite answer set and to do so at the first decode position. This observation turns model training from long-horizon language generation into classification with a language model head. When reinforcement learning (RL) is appropriate, the episode has one action and one terminal reward: emit one label token. The resulting problem is a contextual bandit, avoiding credit assignment over reasoning traces, verbosity rewards, answer extraction, and most format failures. The complete class distribution remains available from the label logits, so a sampled token is used only for training and the serving path need not sample at all. We describe the objective, label-token design, reward and data construction, calibration, and deployment checks. We also state the important negative result: tuning is optional. A general instruction model that already ranks the choices correctly can be used unchanged by reading its next-token log-probabilities. Fine-tuning is primarily justified when the deployment requires a more stable first-token interface, a domain-specific decision boundary, or improved calibration—not merely to make the model look specialized.

1 The training target is a classifier

The companion hearim paper [1] shows how an ordinary autoregressive model can implement the Jev `/v1/systemone` contract: compile a typed question into a labeled choice, read the candidate labels’ next-token log-probabilities, and conditionally normalize them. If $h_\theta(x)$ is the model state after prompt x and $z_\theta(\ell \mid x)$ is the logit for label token ℓ , then the answer distribution is

$$q_\theta(k \mid x, A) = \frac{\exp z_\theta(\ell_k \mid x)}{\sum_{j \in A} \exp z_\theta(\ell_j \mid x)}, \quad (1)$$

where A is the set of answers declared by the request. Choice, ordinal score, and boolean (*noul*) questions are all reductions of this one categorical distribution.

Equation 1 is also the correct training abstraction. The model is a classifier whose class head happens to be selected rows of its language-model vocabulary matrix. Training it to produce explanations, JSON, or a chain of thought solves a larger problem than the API asks for and introduces irrelevant failure modes. The desired completion is exactly one token; EOS may follow, but it is outside the decision itself.

This design has a practical precedent. Kakao’s Kanana Safeguard 8B is a causal language model trained to classify a user utterance or assistant response and emit a single token such as `<SAFE>` or `<UNSAFE-S4>`; its published inference example sets `max_new_tokens=1`, and its evaluation uses the

first output token [2]. A Jev-like model generalizes the same interface from a fixed safety taxonomy to a request-defined taxonomy.

KT’s Llama SafetyGuard Content Binary model is an even closer example of the probability interface: it declares itself a text classifier, reserves the single tokens `<SAFE>` and `<UNSAFE>`, serves one token, and applies a configurable threshold to their two logits [3]. Its streaming deployment repeatedly classifies the growing response prefix; this changes the input schedule, not the one-token decision. Both guard models are fixed-taxonomy classifiers, whereas a Jev-like model receives the candidate meanings in each request. The distinction affects data diversity and label permutation, but not the central architecture: a causal LM can act as a classifier by making its first-token vocabulary rows the class head.

2 Labels and prompt contract

Let the serving prompt end immediately before the label position:

```
State: ...
Question: Which handling policy applies?
<answer-a> = refund
<answer-b> = replace
<answer-c> = escalate
Answer:
```

The literal spelling is not important. The following invariants are.

1. Every label must be exactly one token in the answer context. Verify tokenization after the delimiter; do not infer it from how the label looks.
2. The entire candidate alphabet must be available at training and serving time. Labels are assigned independently of answer meaning and their order is permuted during training to prevent a semantic prior from attaching to a position.
3. The production chat template, system text, delimiter, and answer marker must match training. The loss is applied only at the first answer position.
4. Inference reads all candidate logits. It does not estimate a distribution by repeatedly sampling the one-token answer.

Existing stable tokens such as digits or letters are usually sufficient and require no tokenizer or embedding change. Dedicated tokens such as `<answer-a>` are optional. They are useful when no sufficiently large alphabet remains single-token at the relevant boundary, when label strings collide with normal-language priors, or when a hard protocol boundary is operationally valuable. Otherwise they add tokenizer migration, embedding initialization, checkpoint compatibility, and serving configuration work without changing the statistical problem. If added, the tokenizer and model vocabulary must be resized together, every new token must receive a trainable embedding/output row, and the complete deployment stack must be verified against that exact tokenizer revision.

3 Choose the least training that solves the problem

There are three increasingly invasive regimes.

No tuning. First measure an ordinary instruction model using the exact serving prompt. If class accuracy, negative log-likelihood, calibration, candidate mass, and label-order stability meet the product target, use it unchanged. The model already computed the needed distribution; changing its weights creates no intrinsic benefit.

Supervised one-token fine-tuning. When a gold class y is available for every example, masked next-token cross-entropy is the direct objective

$$\mathcal{L}_{\text{SFT}}(\theta) = -\log q_{\theta}(y \mid x, A). \quad (2)$$

It is normally simpler and more sample-efficient than on-policy RL for the same fixed labels. Training examples should contain the production prompt, one answer token, and no rationale; all prompt-token losses are masked. SFT is the right first tool when the requested behavior can be written down as a labeled classification corpus.

One-step RL. RL is attractive when correctness is supplied by an executable verifier, policy rule, downstream outcome, preference comparison, or other reward that is easier to evaluate than to label densely. It is also a natural final alignment stage after SFT. Crucially, a Jev-like episode does not need a reasoning trajectory:

$$x \longrightarrow a \in A \longrightarrow r(x, a).$$

There is one state, one sampled label token, and one terminal reward. This is a contextual bandit. Policy-gradient training therefore has no temporal credit-assignment problem:

$$\nabla_{\theta} J = \mathbb{E}_{x, a \sim q_{\theta}} [(r(x, a) - b(x)) \nabla_{\theta} \log q_{\theta}(a \mid x, A)], \quad (3)$$

where $b(x)$ is a learned value, a group-relative baseline, or the mean reward of several sampled labels. PPO [4] or a group-relative update can implement this objective, but the important optimization is not the algorithm’s name: it is deleting every action after the first token.

For a correct class y , a minimal reward is

$$r(x, a) = \mathbf{1}[a = y] - \lambda_{\text{invalid}} \mathbf{1}[a \notin A]. \quad (4)$$

Cost-sensitive classification can substitute a reward matrix $R(y, a)$; ordinal questions can reward near misses smoothly; a noul reward can be a proper score from a delayed binary outcome. When probability quality matters, a proper scoring rule should also be optimized or used for model selection. Rewarding only argmax correctness does not by itself calibrate the remaining logits.

The update should include a modest KL penalty to a reference model when preserving general behavior matters. Candidate-masked sampling makes every action syntactically valid and improves exploration, but it changes the training policy’s probability space. Deployments that score unmasked logits must additionally penalize probability mass outside A or mix in examples trained on the full vocabulary. Otherwise a model may classify correctly only when a decoder grammar rescues it.

4 Efficient recipe

An efficient training run follows a short loop.

1. Freeze the production prompt contract and construct a stable single-token alphabet large enough for the maximum class count.

2. Build domain examples as (x, A, y) or (x, A, R) and split them by underlying entity or scenario, not by surface paraphrase. Randomize label assignment and answer order independently for every presentation.
3. Establish the untuned baseline. Record accuracy, macro-F1, NLL, Brier score, expected calibration error, candidate mass, invalid-token mass, and permutation variance.
4. If explicit labels exist, perform masked one-position SFT. Parameter-efficient adapters can reduce memory and make rollback simple; full tuning is not required by the interface.
5. If rewards are programmatic, preference-based, or downstream, continue with one-step RL. Sample only the label position, compute terminal reward, and update against a frozen reference with KL control. Mix difficult and ambiguous examples rather than spending compute generating rationales.
6. Calibrate on held-out data. A scalar temperature T fitted to NLL gives $q_T(k) \propto \exp(z_k/T)$ without changing argmax accuracy. Maintain separate calibration profiles for every model, tokenizer, prompt template, and scoring mode.
7. Reject the new checkpoint unless it improves the target metric while passing token-boundary, label-permutation, invalid-mass, and out-of-domain regression tests.

For K classes, naive on-policy exploration becomes sparse as K grows. If the correct answer is already known, Equation 2 uses that information exactly and should not be replaced by sampling. If only a black-box reward is available, multiple label samples or enumerating all K label rewards can reduce variance; when all rewards can be evaluated, the expected reward $\sum_k q_\theta(k | x, A) R(x, k)$ can be differentiated directly. Calling the latter procedure “RL” is organizationally harmless, but computationally it has recovered a full-information classification update.

5 What tuning is—and is not—for

Fine-tuning cannot make a serving API reveal logits it withholds, cannot repair a tokenizer mismatch outside the trained context, and cannot turn an argmax-only endpoint into a calibrated probability service. Those are interface constraints. Nor should fine-tuning be used merely to force a model to narrate less: raw completions, constrained decoding, or direct logit access may already provide the needed surface.

The defensible reasons to tune are narrower:

- raise task accuracy on a specific decision boundary;
- concentrate answer probability at the first position and reduce mass on prose, reasoning openers, or malformed labels;
- reduce sensitivity to label permutation and prompt paraphrase;
- fit domain costs or rewards not represented in generic instruction tuning; and
- improve calibration, validated on held-out examples.

Thus output stability is the default justification. Specialization is an option when measurements demand it, not a prerequisite for Jev-compatible serving.

6 Evaluation and release gate

Report more than first-token accuracy. Macro-F1 detects neglected classes; NLL and Brier score test the full distribution; ECE gives a coarse calibration view; candidate mass reveals whether the model prefers non-label text; and label-order permutation variance exposes positional shortcuts. Slice all metrics by question type, class count, language, domain, ambiguity, and prompt length. Evaluate both constrained and unconstrained decoding, while using unconstrained candidate logits for the published Jev distribution.

The release artifact must bind the weights to a tokenizer digest, chat or completion template, delimiter, label alphabet, and calibration temperature. A startup probe should verify that each label is one token in context and that the server returns every candidate log-probability. These checks matter even for a perfectly trained checkpoint: the output protocol is partly a property of the serving stack.

7 Conclusion

The most economical Jev-like model is a classifier presented through an autoregressive interface. If training is needed, supervise or reinforce one decision token, not a generated explanation. One-step RL is especially clean when reward is easier to obtain than a gold label: it reduces the episode to a contextual bandit and makes format compliance part of the same terminal decision. Yet no-training remains a first-class result. General models often rank short labeled choices well enough already, and hearim can read that distribution without changing the model. Dedicated `<answer-a>` tokens, SFT, and RL are tools for measured deficiencies—above all output stability—rather than requirements of the architecture.

References

- [1] J. Jung. *hearim: Turning Generation APIs into Probability Oracles—A Jev-Compatible Reimplementation of Single-Token Label Scoring*. 2026. Companion paper in this repository.
- [2] Kanana Safeguard Team. *Kanana Safeguard*. Kakao, 2025. <https://huggingface.co/kakaocorp/kanana-safeguard-8b>.
- [3] W. Lee, Y. Kim, Y. Park, J. Moon, D. Jeong, and W. Park. Guard Vector: Beyond English LLM Guardrails with Task-Vector Composition and Streaming-Aware Prefix SFT. *arXiv:2509.23381*, 2025. Model card: <https://huggingface.co/K-intelligence/Llama-SafetyGuard-Content-Binary>.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [5] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.